Backend System Design & Architecture

Graniti Vicentia V1 - AI-assisted shop drawing review

> **Core operating principle**
> The AI reads. Evidence qualifies. Deterministic code decides. A reviewer signs off.

Prepared as a backend-focused proposal based on the client architecture document dated 30 July 2026, followed by a fresh verification of official technical sources on 2 August 2026.

Scope: cabinets and countertops; digitally generated PDFs as the primary path; scanned-page fallback; one GV review team; evidence-linked findings; reviewer corrections; no autonomous approval or automatic rule learning.

Proposal statement

Build a workflow-driven modular backend with five explicit trust zones: application control, durable workflow execution, bounded AI extraction, evidence and matching, and isolated deterministic decisioning. PostgreSQL owns business truth; Hatchet owns execution history; S3 owns immutable artifacts.

Research verified: 2 August 2026

Architecture status: recommended V1 target with production-pilot constraints

Contents

> **Reading note**
>
> The document deliberately separates source-supported facts from architecture recommendations. Reference markers [R#] point to official documentation listed in Appendix C; [B1] refers to the supplied GV architecture baseline.

1. Executive architecture decision

The recommended backend is not an autonomous “AI reviewer.” It is a durable document-processing and evidence-validation system in which AI can propose observations but cannot approve evidence, choose tolerances, or issue a manufacturing verdict. Hatchet owns workflow execution state; PostgreSQL owns business truth and package lifecycle.

Decision area	Recommended V1 choice	Reasoning
Business backend	FastAPI modular monolith	Keeps project, package, revision, review and audit logic cohesive while avoiding premature microservices.
Workflow	Hatchet OSS; Hatchet Lite only for local or explicitly low-volume pilot use	Hatchet provides persisted DAG execution, retries, waits and parallel tasks. Its Lite image is documented for development and low-volume workloads. [R1-R3]
Agentic layer	One bounded LangGraph extraction graph	Useful only when fixed vector/OCR/geometry routing cannot resolve an ambiguous region. Checkpoint and interrupt behavior requires idempotent side effects. [R4-R5]
Evidence	Candidate → canonical observation → evidence gate	Raw model or OCR output must never become a verdict operand without schema, unit, authority and corroboration checks.
Matching	Exact identifiers and geometry first; retrieval advisory	Coded identifiers need exact or controlled fuzzy matching before semantic recall. [R9-R11]
Rules	Versioned rules + deterministic applicability resolver	Rule selection must be reproducible and must not be delegated to an LLM.
Verdict	Isolated typed Python service	Exact arithmetic, no eval, no model credentials, no retrieval access and no conversational memory.

> **Primary safety metric**
>
> Critical false-PASS rate. A false FAIL creates review work; a false PASS may become a manufactured error. Automatic PASS is enabled per check type only after held-out gold-set acceptance.

2. Research-backed changes to the baseline

The supplied architecture is directionally strong. Fresh web verification supports its core separation of workflow, bounded extraction, canonical evidence, advisory retrieval and deterministic verdicts. The following refinements make the backend safer and easier to operate.

Baseline position	Refined proposal	Why
Hatchet owns package state	Hatchet owns workflow execution; the application database owns package lifecycle	Business state must remain queryable and portable even if the workflow engine changes.
Hatchet Lite on one VM	Use Lite for local/staging or a bounded low-volume pilot; document the no-HA constraint	Official Hatchet documentation calls the Lite image development and low-volume oriented. [R1]
Agent returns structured output	Use client-side tool schema plus strict Pydantic validation	Nova 2 Lite supports client-side tool calling but its model card lists structured outputs as unsupported. [R6]
Canonical model receives extractor output	Introduce ObservationCandidate before CanonicalObservation	A separate candidate type preserves uncertainty and prevents accidental promotion of raw model output.
Matching uses hybrid retrieval	Retrieval produces MatchCandidate only; deterministic checks or reviewers approve matches	Retrieval may increase recall but must never create authoritative dimensions or item associations.
Evidence gate before verdict	Add deterministic Rule Applicability Resolver before verdict	The architecture needs an explicit non-AI owner for deciding which published rule snapshot applies.
File hashes and S3 originals	Store S3 version ID + SHA-256; optionally use Object Lock for originals/final reports	Versioning preserves prior object versions; Object Lock adds WORM protection where required. [R12-R13]
Logs and task history	Adopt end-to-end OpenTelemetry and domain metrics	Trace package → workflow → task → model call → finding for debugging and release governance. [R14]

What remains intentionally deferred

- General multi-tenant enterprise architecture, custom detector training and full drawing-domain automation.
- GraphRAG, a dedicated vector database, a dedicated lexical search cluster and an MCP integration layer.
- Automatic rule generation, automatic exception learning and autonomous vendor submission.
- High availability and autoscaling until concurrency, recovery time or commercial uptime requirements justify them.

3. Backend trust zones and responsibility boundaries

The backend is divided by authority, not only by technology. Each zone has a strict output contract and explicit “must not own” boundary.

Zone / component	Owns	Must not own
Application & control plane	Authentication, authorization, project/package lifecycle, revisions, reviewer actions, status APIs, audit events	OCR, model reasoning, PASS/FAIL calculations
Hatchet workflow plane	Task scheduling, retries, page parallelism, cancellation, waits, task history	Business truth, semantic interpretation, tolerances
Bounded extraction plane	Tool routing, OCR verification, crop refinement, geometry association, abstention	Rule selection, evidence approval, final match approval, verdicts
Canonical evidence plane	Typed facts, units, polygons, provenance, authority class, evidence status	Hidden prompt assumptions or invented values
Advisory retrieval	Candidate discovery, aliases, typo suggestions, lexical and semantic recall	Authoritative dimensions, exceptions or direct verdict operands
Evidence gate	Eligibility of observations and matches for checking	Inventing missing inputs or resolving numeric disagreements by model preference
Rule resolver	Deterministic selection of applicable published rule snapshots	Generating new rules or changing tolerance
Verdict service	Unit normalization, exact arithmetic and typed operation execution	Models, retrieval, internet, memory or arbitrary code
Reviewer	Final approval, corrections and documented exceptions	Untracked edits or silent rule changes

> **Non-negotiable boundary**
>
> No path exists from VLM output, OCR confidence or retrieval score directly into the verdict service. Only evidence-gate-approved, versioned operands can cross that boundary.

4. Logical backend architecture

The architecture below shows the main authority boundaries. Solid arrows are processing paths; dashed or dotted paths are advisory, audit or telemetry paths.

Architecture invariant	Backend implication
Business truth is independent of orchestration	Package, revision, finding and approval state remain in the application database.
AI creates candidates, not facts	Only normalized and corroborated observations can reach the evidence gate.
Retrieval is advisory	Search results can suggest a match but cannot supply authoritative operands.
Rule applicability is deterministic	The resolver selects a published snapshot from explicit scope fields.
Verdict execution is isolated	No model, retrieval, memory or arbitrary code is available to the verdict process.

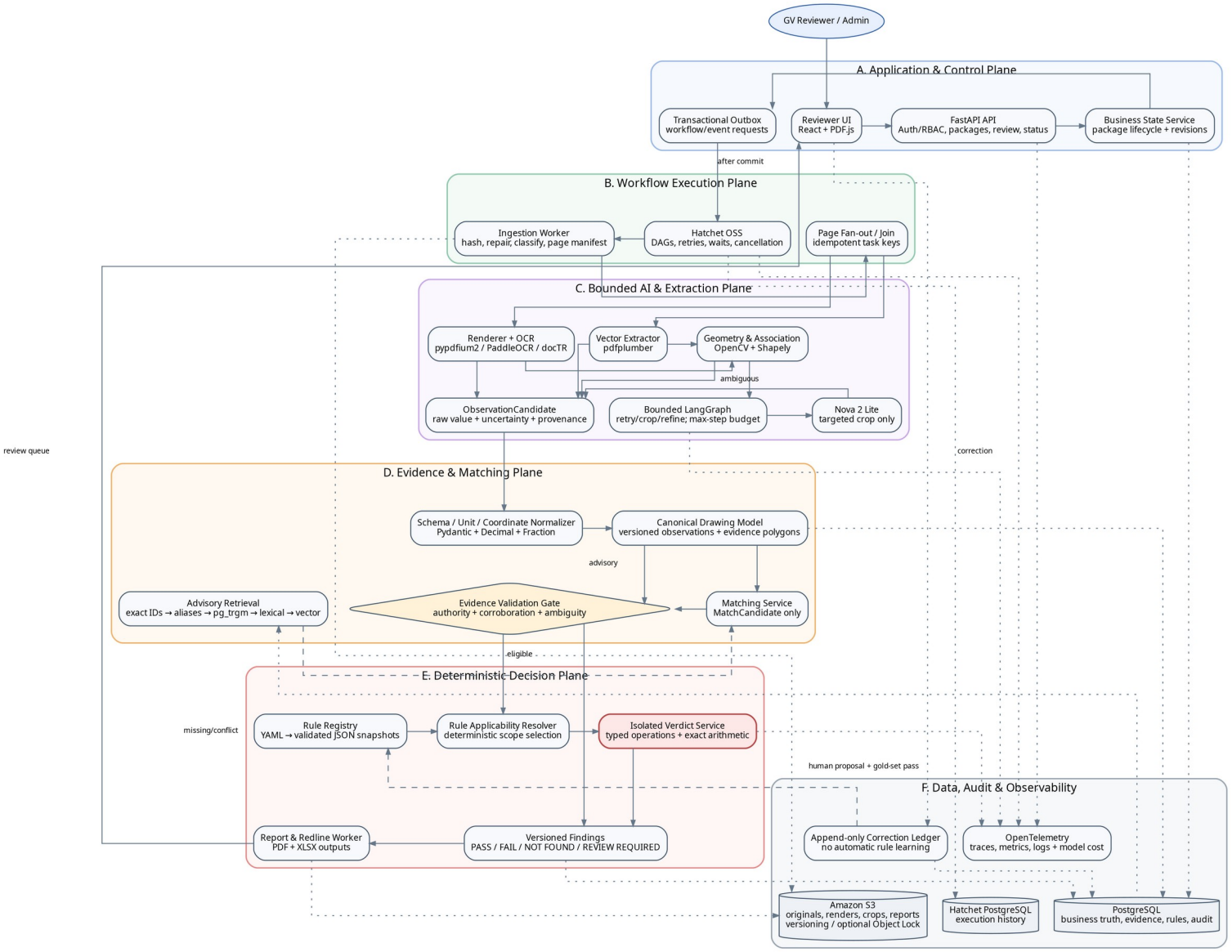

Figure 1. Recommended backend logical architecture. The exact Mermaid source is included in Appendix A.

> **Architecture reading**
> The center flow is: workflow → bounded extraction → canonical evidence → evidence gate → deterministic rule applicability → isolated verdict → versioned finding. Data, corrections and retrieval remain side channels and cannot bypass the gate.

4.1 Component interactions

The HTTP API performs short control-plane work only. Uploads use presigned S3 URLs where possible; CPU-heavy rendering, OCR, geometry and report generation run as background tasks. The API creates or updates business records and then emits an outbox event in the same database transaction. A dispatcher starts Hatchet only after the transaction commits, avoiding a package record without a workflow or a workflow without a package record.

Workers write idempotent, versioned results. They never mutate prior document versions or findings in place. A rerun creates a new extraction run, observation version or finding revision tied to exact code, rule and model versions.

5. End-to-end backend processing flow

The flow is designed to fail safely. Missing evidence becomes NOT FOUND; disagreement or judgment becomes REVIEW REQUIRED; PASS and FAIL require validated operands and a published rule snapshot.

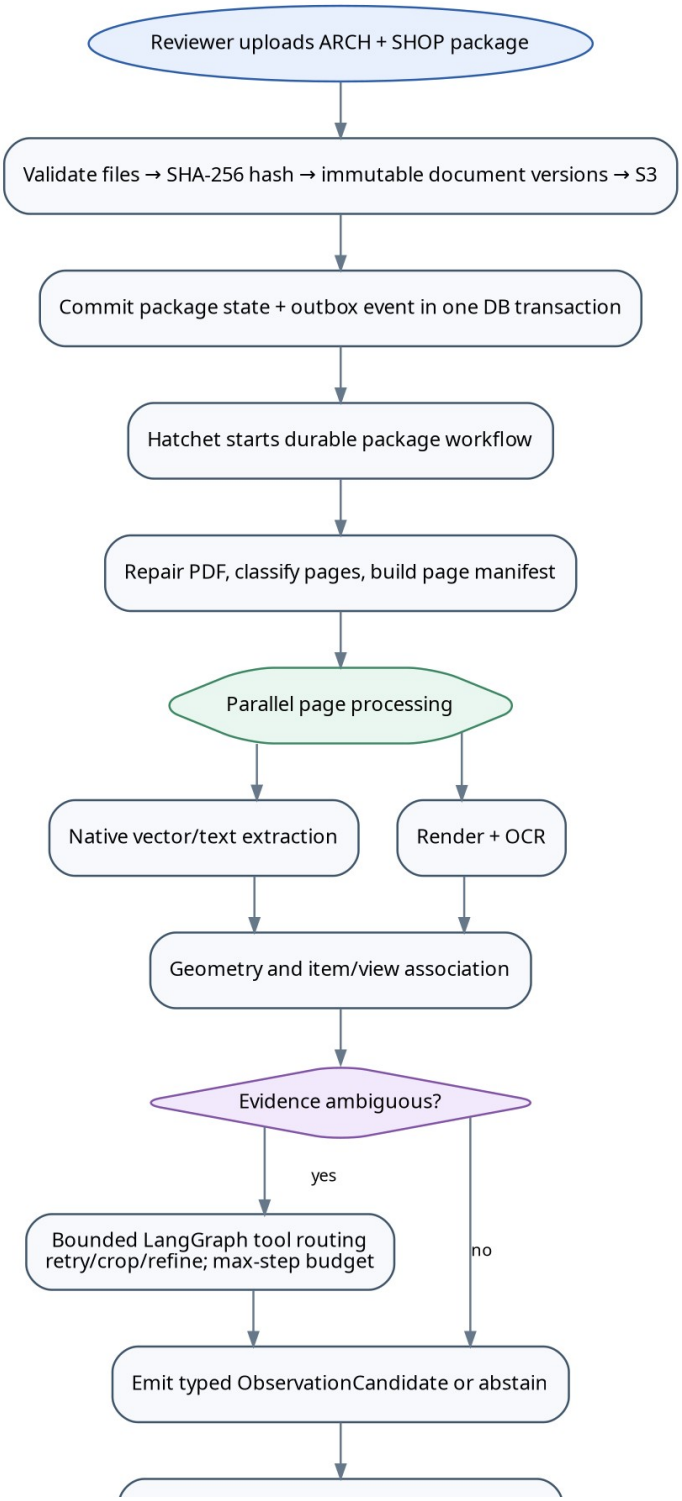

Figure 2A. Processing flow - ingestion, parallel extraction and candidate formation.

5. End-to-end backend processing flow (continued)

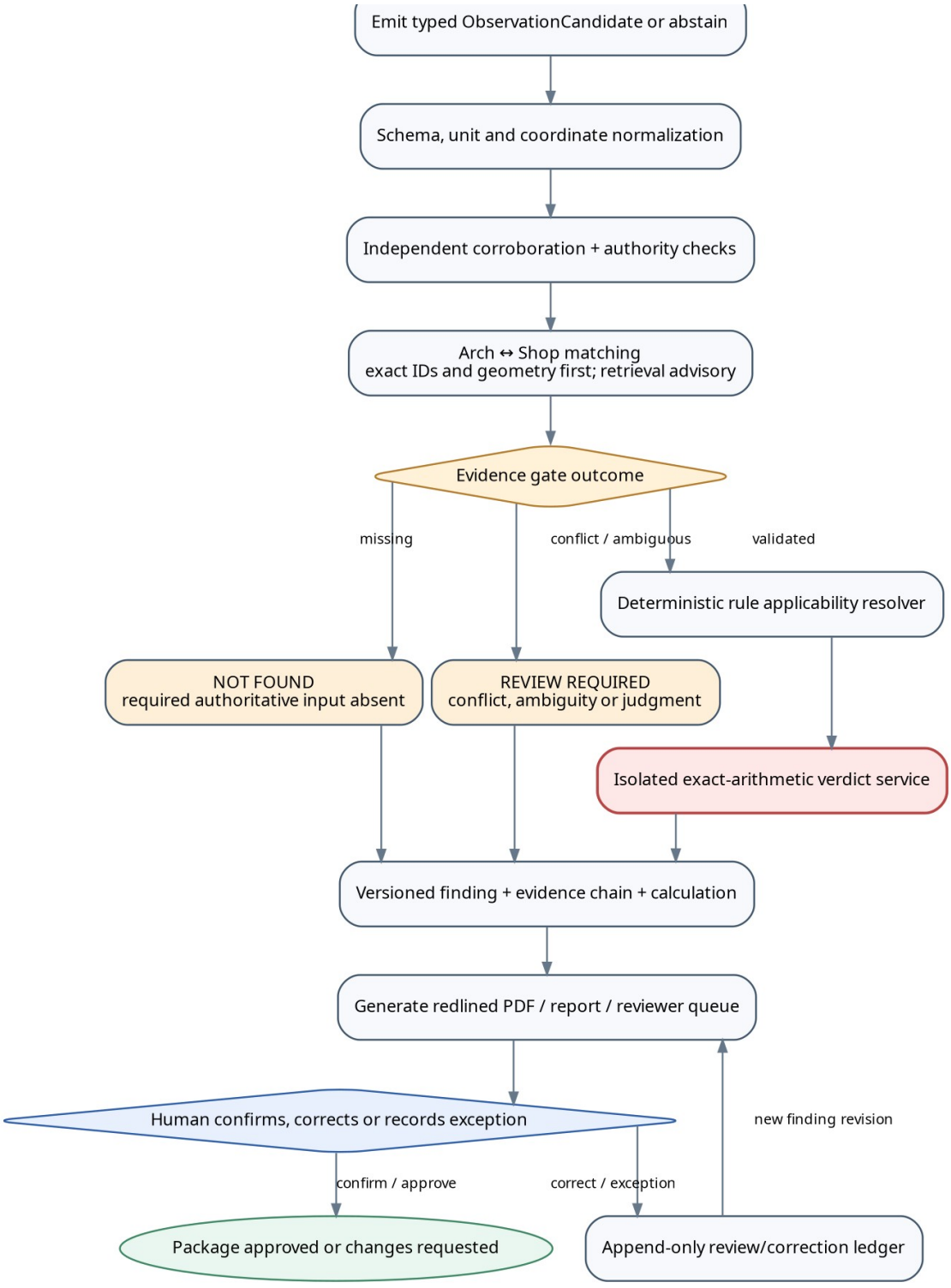

Figure 2B. Processing flow - evidence gate, deterministic verdict, outputs and review ledger.

Outcome	Produced by	Meaning
PASS	Verdict service	Validated expected and actual operands satisfy the published rule

Outcome	Produced by	Meaning
		and tolerance.
FAIL	Verdict service	Validated operands violate the published rule or tolerance.
NOT FOUND	Evidence gate	A required authoritative source, value, unit or approved match is absent.
REVIEW REQUIRED	Evidence gate / reviewer	Evidence conflicts, association is ambiguous, semantics require judgment or a documented exception is needed.

6. AI and extraction subsystem

6.1 Fixed extraction before agentic extraction

Most pages should follow a fixed, cheaper and more reproducible route: PDF repair → native vector/text extraction → stable rendering → OCR if needed → geometry association. pdfplumber is explicitly oriented toward detailed access to characters, lines and rectangles and works best on machine-generated PDFs. [R17] PaddleOCR and docTR remain separate readers so critical disagreement can be detected instead of silently averaged. [R18-R19]

6.2 Bounded LangGraph role

LangGraph is used only when the current evidence state is ambiguous. It may select a permitted tool, refine a crop, request one verification route and abstain. It may not select rules, alter tolerances, approve a package or write a PASS/FAIL result. Checkpoints provide fault tolerance; interrupt resumes restart the node, so every side effect before an interrupt must be idempotent. [R4-R5]

Guardrail	V1 policy
Maximum graph steps	6-8 per ambiguous region
OCR retries	Maximum 2, with versioned attempt records
VLM calls	1 primary targeted-crop call and at most 1 escalation
Context	Crop + bounded nearby text/geometry; no full package by default
Output	Tool-schema payload → Pydantic validation → ObservationCandidate
Numeric disagreement	Never resolve by model preference; mark CONFLICTING
Terminal states	Candidate with provenance or explicit abstention
Budget	Per-package call and token ceiling; overflow becomes REVIEW REQUIRED

6.3 Nova 2 Lite adapter

Nova 2 Lite is a cost-efficient multimodal model positioned for document processing. Its model card lists client-side tool calling as supported and structured outputs as unsupported. The adapter should therefore define a strict tool input schema, validate the returned payload, reject unknown fields and persist the prompt/template/model identifiers. [R6]

> **Model trust policy**
> Model confidence is diagnostic metadata, not evidence authority. A high-confidence VLM result is still a candidate until corroborated or human-confirmed.

7. Canonical evidence, matching and retrieval

7.1 Two-stage observation model

Type	Purpose	Representative fields
ObservationCandidate	Preserves raw extractor output and uncertainty	extractor, version, raw_text, parsed_value, unit_guess, semantic_guess, polygon, confidence, ambiguity_flags
CanonicalObservation	A versioned, normalized fact eligible for matching or review	document_version_id, page, item/view, semantic_type, normalized_value, canonical_unit, evidence_crop_uri, status, authority_class
VerdictOperand	A sealed input created only after the evidence gate	observation IDs, approved match ID, canonical values/units, rule snapshot ID, evidence hashes

7.2 Evidence states

State	May enter verdict?	Meaning
RAW_CANDIDATE	No	One unverified extraction route produced it.
CORROBORATED	Yes	Independent routes agree and the semantic association is valid.
HUMAN_CONFIRMED	Yes	A reviewer explicitly confirmed the value and evidence location.
CONFLICTING	No	Readers, units or associations disagree.
REJECTED	No	The candidate was invalidated.

7.3 Exact-first matching

Dense embeddings alone are unsafe for coded identifiers such as X-223 and X-233. PostgreSQL pg_trgm supports indexed alphanumeric similarity and is useful for controlled typo candidate generation; pgvector documents hybrid use with PostgreSQL full-text search and Reciprocal Rank Fusion. BM25S is a lightweight Python BM25 implementation, but it should remain an optional V1 lane rather than a required service. [R9-R11]

Priority	Lane	Backend rule
1	Exact normalized identifier	Pin exact code before any ranking.
2	Alias table	Resolve known forms such as X223, X-223 and X 223.

Priority	Lane	Backend rule
3	Hard filters	Project, revision, category, document role and view type.
4	Geometry / view relationship	Prefer evidence-linked association over semantic similarity.
5	pg_trgm	Suggest likely OCR variants; never silently accept.
6	Lexical BM25 / PostgreSQL FTS	Recall rare codes, terminology and exact phrases.
7	Dense pgvector	Recall semantic equivalents and related descriptions.
8	Fusion	Combine rankings while keeping exact ID pinned.

8. Deterministic rules and verdict service

8.1 Rule lifecycle

Rules are authored in reviewable YAML, validated against Pydantic and JSON Schema, and published as immutable JSON snapshots. Each finding stores the exact rule snapshot ID. Changes require human approval and a complete gold-set regression; they never arise directly from reviewer corrections.

Rule concern	V1 design
Applicability	Deterministic resolver using product category, item type, material/configuration, semantic type, project scope and effective version.
Execution	Small typed operation registry; no eval, no arbitrary formulas.
Arithmetic	Fraction for architectural fractions; Decimal for canonical values and conversions.
Units	Millimetres by default; unknown or ambiguous units cannot enter verdict.
Missing evidence	NOT FOUND.
Ambiguous evidence	REVIEW REQUIRED.
Versioning	Rule snapshot, operation version and engine version stored with every finding.

8.2 Verdict-service contract

Input	Output
Sealed rule snapshot	Outcome and reason code
Expected and actual VerdictOperand	Normalized operands and units
Approved match reference	Calculation trace and tolerance
Evidence hashes and observation IDs	Engine version and deterministic input hash

> **Isolation requirement**
> The verdict process has no Bedrock credentials, no vector/BM25 access, no historical correction memory, no rule-authoring route, no arbitrary code execution and no general outbound internet access.

Initial typed operations: exists; equals; within_tolerance; minimum; maximum; between; one_of; contains; count_equals; conditional_required; difference_between. New operations are code-reviewed and tested rather than supplied as executable rule text.

9. Business state, workflow state and idempotency

9.1 Package state machine

State	Entry condition	Next normal state
CREATED	Package record exists	UPLOADING
UPLOADED	Required source files have immutable versions	INGESTING
INGESTING	Workflow accepted	EXTRACTING
EXTRACTING	Page manifest created	MATCHING
MATCHING	Canonical observations available	VALIDATING_EVIDENCE
VALIDATING_EVIDENCE	Match candidates resolved	RUNNING_CHECKS
RUNNING_CHECKS	Eligible operands and rule snapshots available	GENERATING_OUTPUTS
GENERATING_OUTPUTS	Findings persisted	AWAITING_REVIEW
AWAITING_REVIEW	Reviewer queue ready	APPROVED / CHANGES_REQUESTED

Side states are FAILED_RETRYABLE, FAILED_PERMANENT, NEEDS_INPUT, CANCELLED and SUPERSEDED. A new document revision never overwrites an old version; it supersedes the prior package revision and starts a new workflow run.

9.2 Idempotency keys

Every expensive or externally visible task uses a stable key derived from document_version_id + page_number/region_id + task_type + extractor_version + configuration_hash. A unique database constraint makes retries safe. Hatchet durable tasks should contain deterministic control logic and delegate database or external API side effects to child tasks, matching Hatchet's official durability guidance. [R2]

9.3 Transactional outbox

The API writes package state and an outbox row in one PostgreSQL transaction. A dispatcher publishes the workflow request and marks the outbox row delivered. This prevents dual-write failures and makes workflow start replayable. This is an architecture recommendation rather than a claim from the supplied baseline.

9.4 Retry policy

Failure class	Policy
Transient S3/Bedrock/network error	Bounded exponential backoff with jitter and task timeout.
Malformed PDF	One repair path; then FAILED_PERMANENT or reviewer replacement request.

Failure class	Policy
OCR disagreement	No auto-resolution; create CONFLICTING evidence.
Model schema failure	One repair/second call maximum; then abstain.
Unknown unit	Block verdict and route to REVIEW REQUIRED.
Worker crash	Retry idempotent task; reuse completed artifacts and checkpoints.

10. Data model and API contracts

10.1 Core PostgreSQL aggregates

Aggregate	Core records
Project & package	projects, packages, package_revisions, package_state_events
Documents	documents, document_versions, pages, source_artifacts
Execution	workflow_runs, task_runs, extraction_runs, model_invocations
Evidence	observation_candidates, canonical_observations, evidence_artifacts, evidence_relationships
Drawing model	drawing_items, drawing_views, item_identifiers, aliases
Matching	match_candidates, approved_matches, match_review_events
Rules	rule_definitions, rule_snapshots, rule_applicability_scopes
Checks	check_runs, verdict_inputs, findings, finding_evidence
Review & governance	review_sessions, review_actions, correction_ledger, approvals, exceptions
Evaluation	gold_sets, gold_cases, evaluation_runs, metric_results

10.2 API shape

API group	Representative operations
Packages	create package; request upload URLs; finalize upload; get state; cancel; supersede
Documents	list versions/pages/artifacts; obtain short-lived evidence URL
Findings	list/filter findings; get calculation/evidence chain; export report
Review	confirm; correct; record exception; approve; request changes

API group	Representative operations
Rules	list snapshots; validate proposal; publish after authorization and regression
Operations	workflow status; retry permitted task; health/readiness; metrics

> **API rule**
> The API never accepts a client-provided PASS/FAIL calculation. Reviewer actions reference server-side finding revisions and produce append-only review events.

11. Security, privacy and audit controls

The drawings are proprietary source material and model input must be treated as untrusted. Security controls should be enforced through network, IAM, database roles and immutable audit records rather than prompt instructions alone.

Control area	Recommended implementation
Object storage	S3 versioning; SHA-256 in PostgreSQL; SSE-KMS; short-lived presigned URLs; optional Object Lock for originals/final reports. [R12-R13]
Bedrock data boundary	Dedicated worker IAM role; targeted crops only; persist model/call metadata; no verdict credentials. AWS states model providers cannot access Bedrock customer prompts or completions. [R7]
Network	Private subnets and VPC endpoints/PrivateLink when production scale justifies it. AWS documents private connectivity for Bedrock-related data paths. [R8]
Database access	Separate application, worker, read-only reporting and verdict roles; least privilege; encrypted connections; audit sensitive reads.
Container isolation	Verdict container with no egress except required database endpoint and no model/retrieval secrets.
Authorization	Project-scoped RBAC; only authorized reviewers can confirm evidence or approve packages; only rule administrators publish snapshots.
Prompt injection	Treat drawing text as data, not instructions; fixed system prompts and tool allow-list; no dynamic external tools.
Audit	Append-only state, rule, finding, review, exception and download events with actor, timestamp and request/trace ID.
Retention	Explicit source, crop, model-call and log retention schedules; lifecycle policies for noncurrent S3 versions.

Threat-focused checks

- A numerically plausible but incorrect extraction is blocked by independent verification and evidence polygons.
- A wrong item/view association is blocked by exact identifiers, geometry and reviewer confirmation for ambiguity.
- A VLM hallucination remains a non-authoritative candidate and cannot cross the evidence gate.
- A malicious rule expression cannot execute because operations are selected from a typed registry.
- A retrieved historical correction cannot contaminate truth because the verdict service has no retrieval access.

12. Observability and release metrics

OpenTelemetry provides a vendor-neutral way to emit traces, metrics and logs. Each package receives a trace ID propagated through API, Hatchet tasks, extraction attempts, Bedrock calls, evidence validation, verdicts and report generation. [R14]

Telemetry	Required dimensions / examples
Traces	package_id, document_version_id, workflow_run_id, task_run_id, page/region, extractor version, rule snapshot
Metrics	task duration, retry count, OCR disagreement rate, VLM call rate, token/cost estimate, queue wait, reviewer minutes
Safety metrics	critical false-PASS, evidence polygon correctness, numeric/unit exact match, identifier match precision
Coverage metrics	PASS/FAIL/NOT FOUND/REVIEW REQUIRED distribution, automation coverage by check type
Audit logs	rule publication, package approval, exception creation, artifact download and correction events

Release gates

Gate	Required behavior
OCR disagreement	Never auto-resolve; REVIEW REQUIRED.
Unknown unit	Cannot enter verdict.
Missing approved source	NOT FOUND.
Advisory retrieval result	Cannot become a verdict operand.
Rule change	Human approval + full gold-set regression.
New automatic PASS check type	Separate held-out acceptance decision.
Evidence localization	Finding page and polygon meet release threshold.

> **Operational definition of success**
> Optimize reviewer minutes and automation coverage only after false-PASS, evidence localization, numeric/unit accuracy and match precision meet their release gates.

13. V1 deployment and scale-out path

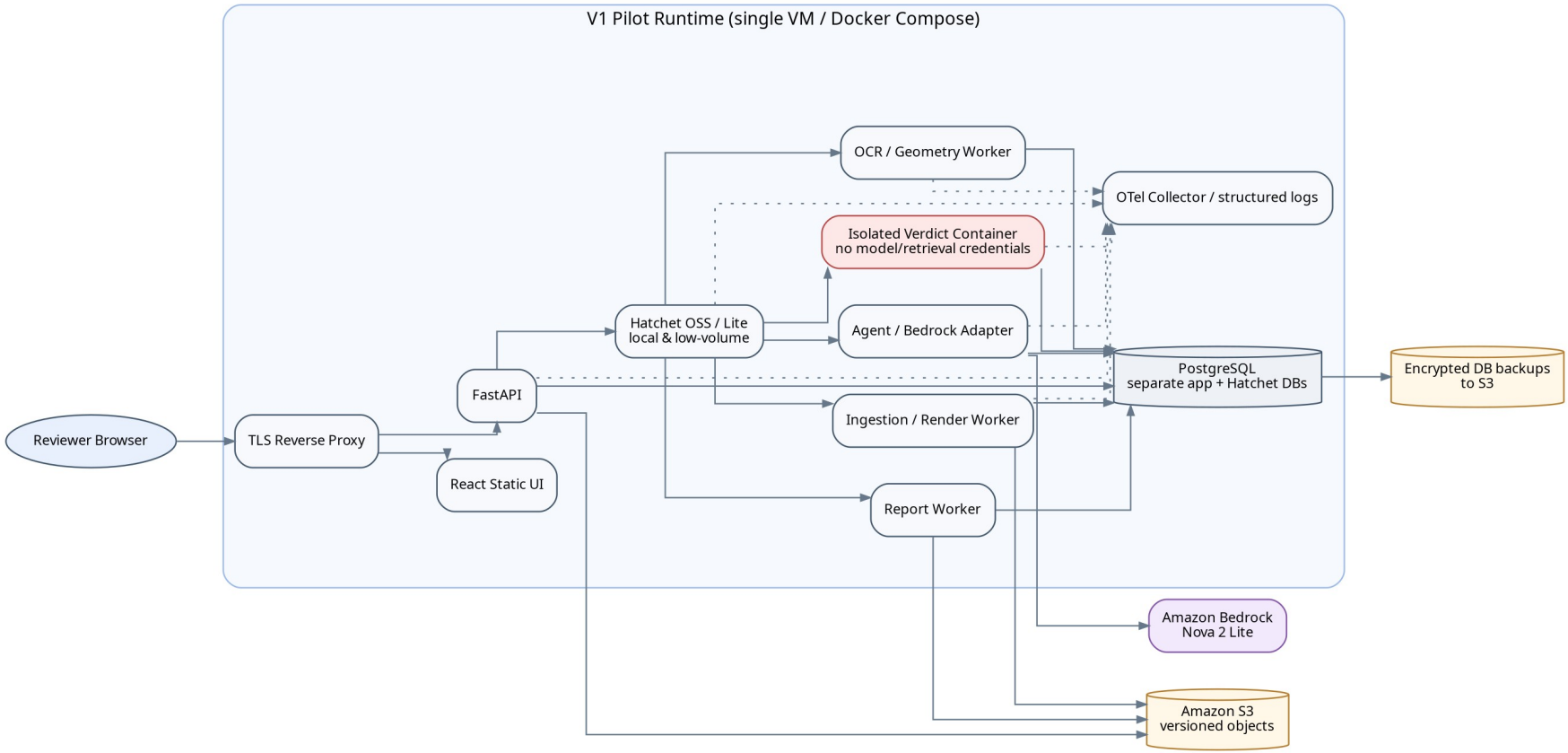

Figure 3. V1 pilot deployment. The verdict process is separately isolated even when other components share one VM.

13.1 V1 pilot topology

A single 8 GB VM can support a controlled pilot only with strict worker concurrency. Rendering, OCR, PostgreSQL and Hatchet compete for memory, so the deployment should cap concurrent packages, rendered pages, OCR processes and VLM requests. Originals and reports remain in S3; encrypted database backups are copied off the VM.

Hatchet Lite is suitable for local development and documented low-volume use. For a production pilot, explicitly accept its availability limits or use a fuller Hatchet OSS/managed deployment. [R1]

13.2 Scale-out triggers

Measured trigger	Upgrade
Sustained worker contention or more than ~5-10 concurrent packages	Separate API, ingestion/render, OCR/geometry, agent and report workers.
Database downtime becomes commercially material	Move application PostgreSQL to RDS with automated backups and PITR. RDS PostgreSQL supports Multi-AZ deployments. [R15]
Formal high availability requirement	ECS/Fargate services plus Multi-AZ RDS; Fargate provides isolated task boundaries without managing servers. [R16]
Workflow recovery or code migration becomes a recurring risk	Evaluate Temporal or a production-grade managed workflow service using measured requirements.
pgvector impacts transactional workload	Move vector retrieval to a dedicated service.
Lexical corpus/latency outgrows embedded BM25	Introduce OpenSearch or another managed lexical service.
Managed VLM spend exceeds GPU economics	Benchmark a self-hosted VLM through a separately isolated inference service.

14. Implementation plan and client proposal

14.1 Build in risk order

Phase	Build	Exit gate
0. Gold set	Annotate representative cabinet/countertop packages: values, units, IDs, polygons, matches and expected findings	Held-out ground truth exists and review policy is agreed.
1. Small core loop	Local upload → render/vector/OCR → evidence → one deterministic check	Exact values and evidence locations are measurable.
2. Canonical evidence	Candidate and canonical schemas, coordinate transforms, immutable artifacts	No raw model output can reach checking.
3. Gate + rules	Evidence eligibility, rule snapshots, applicability resolver and typed operations	Unsupported inputs become NOT FOUND or REVIEW REQUIRED.
4. Bounded agent	Add LangGraph around ambiguous regions only	Improves accuracy, cost or reviewer time versus fixed routing.
5. Matching/retrieval	Exact IDs, aliases, geometry, pg_trgm, then lexical/vector lanes if justified	Match precision improves without increasing false PASS.
6. Durable platform	Hatchet workflows, retries, page parallelism, outbox and observability	Measured core runs reliably as jobs.
7. Reviewer product	Evidence-linked findings, redlines, correction ledger and approvals	Reviewer validates every decision quickly and auditably.

14.2 Client-facing proposal structure

Proposal stage	Client commitment	Engineering deliverable
Discovery and gold-set sprint	Provide representative packages, rules and reviewer time	Benchmark report, error taxonomy, V1 check list and acceptance thresholds
Accuracy core	Approve data contracts and safety outcomes	End-to-end prototype with evidence-linked deterministic checks
Production pilot	Named reviewers and controlled package volume	Authenticated UI, durable workflow, audit, reports, backups and monitoring
Scale decision	Confirm concurrency, SLA, security and integration needs	Measured migration plan to RDS/ECS/managed workflow or dedicated search

Appendix A. Mermaid logical architecture source

This source is the portable architecture-as-code definition corresponding to Figure 1.

```
flowchart TB
  U[GV Reviewer / Admin] --> UI[Reviewer UI - React + PDF.js]

  subgraph CONTROL[Application and Control Plane]
    UI --> API[FastAPI API - Auth/RBAC, packages, review, status]
    API --> STATE[Business State Service]
    STATE --> OUTBOX[Transactional Outbox]
  end

  subgraph WORKFLOW[Workflow Execution Plane]
    OUTBOX -->|after DB commit| H[Hatchet OSS]
    H --> INGEST[Ingestion Worker]
    INGEST --> FAN[Page Fan-out / Join]
  end

  subgraph EXTRACT[Bounded AI and Extraction Plane]
    FAN --> VEC[pdfplumber vector extraction]
    FAN --> OCR[pypdfium2 + PaddleOCR]
    OCR --> VERIFY[docTR verifier - critical/conflicting only]
    VEC --> GEOM[OpenCV + Shapely geometry]
    OCR --> GEOM
    GEOM -->|ambiguous only| LG[Bounded LangGraph]
    LG -->|targeted crop| NOVA[Amazon Nova 2 Lite]
    VEC --> CAND[ObservationCandidate]
    OCR --> CAND
    VERIFY --> CAND
    GEOM --> CAND
    NOVA --> CAND
  end

  subgraph EVIDENCE[Evidence and Matching Plane]
    CAND --> NORM[Schema, unit and coordinate normalizer]
    NORM --> CDM[Canonical Drawing Model]
    CDM --> MATCH[Matching Service]
    RET[Exact IDs -> aliases -> pg_trgm -> lexical -> vector] -. advisory .-> MATCH
    MATCH --> GATE{Evidence Validation Gate}
    CDM --> GATE
  end

  subgraph DECISION[Deterministic Decision Plane]
    RULES[Versioned YAML -> validated JSON rules] --> RESOLVE[Rule Applicability Resolver]
    GATE -->|validated evidence| RESOLVE
    RESOLVE --> VERDICT[Isolated Exact-Arithmetic Verdict Service]
    GATE -->|missing / conflict / ambiguity| FIND[Versioned Finding]
    VERDICT --> FIND
    FIND --> REPORT[Report + Redline Worker]
  end

  subgraph DATA[Data, Audit and Observability]
    PG[(PostgreSQL - business truth)]
    HDB[(Hatchet PostgreSQL - execution state)]
    S3[(Amazon S3 - originals, crops, reports)]
    LEDGER[Append-only Correction Ledger]
    OTEL[OpenTelemetry]
  end

  STATE --> PG
  H --> HDB
  INGEST --> S3
  CDM --> PG
  FIND --> PG
  REPORT --> S3
  REPORT --> UI
  UI --> LEDGER
  LEDGER --> PG
  LEDGER -. human proposal + gold-set pass .-> RULES
  API -. telemetry .-> OTEL
  H -. telemetry .-> OTEL
  LG -. telemetry .-> OTEL
  VERDICT -. telemetry .-> OTEL
```

Appendix B. Mermaid processing-flow source

This source is the portable processing-flow definition corresponding to Figure 2.

```
flowchart TD
  A([Reviewer uploads ARCH + SHOP package])
  B[Validate files, hash, create immutable document versions, store in S3]
  C[Commit package state and outbox event in one DB transaction]
  D[Hatchet starts durable package workflow]
  E[Repair PDF, classify pages, build page manifest]
  F{{Parallel page processing}}
  G[Native vector / text extraction]
  H[Render + OCR]
  I[Geometry and item/view association]
  J{Evidence ambiguous?}
  K[Bounded LangGraph: retry, crop, refine, max-step budget]
  L[Emit typed ObservationCandidate or abstain]
  M[Schema, unit and coordinate normalization]
  N[Independent corroboration and authority checks]
  O[Arch-to-Shop matching: exact IDs and geometry first]
  P{Evidence gate outcome}
  Q[NOT FOUND]
  R[REVIEW REQUIRED]
  S[Deterministic rule applicability resolver]
  T[Isolated exact-arithmetic verdict service]
  U[Versioned finding + evidence chain + calculation]
  V[Generate redlined PDF, report and reviewer queue]
  W{Human review}
  X([Package approved or changes requested])
  Y[Append-only correction / exception ledger]

  A --> B --> C --> D --> E --> F
  F --> G --> I
  F --> H --> I
  I --> J
  J -->|no| L
  J -->|yes| K --> L
  L --> M --> N --> O --> P
  P -->|missing| Q --> U
  P -->|conflict / ambiguous| R --> U
  P -->|validated| S --> T --> U
  U --> V --> W
  W -->|confirm / approve| X
  W -->|correct / exception| Y --> V
```

Appendix C. Research references

Baseline and official sources reviewed for this backend proposal. Web references were re-verified on 2 August 2026.

[B1] Graniti Vicentia, “V1 Agentic Shop Drawing Review Architecture,” supplied 17-page client document, research date 30 July 2026.

[R1] Hatchet, “Hatchet Lite Deployment.” https://docs.hatchet.run/self-hosting/hatchet-lite

[R2] Hatchet, “Durable Tasks.” https://docs.hatchet.run/v1/durable-tasks

[R3] Hatchet, “DAGs as Durable Workflows.” https://docs.hatchet.run/v1/directed-acyclic-graphs

[R4] LangChain, “LangGraph Persistence.” https://docs.langchain.com/oss/python/langgraph/persistence

[R5] LangChain, “LangGraph Interrupts.” https://docs.langchain.com/oss/python/langgraph/interrupts

[R6] AWS, “Nova 2 Lite - Amazon Bedrock model card.” https://docs.aws.amazon.com/bedrock/latest/userguide/model-card-amazon-nova-2-lite.html

[R7] AWS, “Data protection in Amazon Bedrock.” https://docs.aws.amazon.com/bedrock/latest/userguide/data-protection.html

[R8] AWS, “Protect your data using Amazon VPC and AWS PrivateLink - Amazon Bedrock.” https://docs.aws.amazon.com/bedrock/latest/userguide/usingVPC.html

[R9] PostgreSQL, “pg_trgm - support for similarity of text using trigram matching.” https://www.postgresql.org/docs/17/pgtrgm.html

[R10] pgvector project, “Hybrid Search.” https://github.com/pgvector/pgvector

[R11] BM25S project documentation. https://bm25s.github.io/

[R12] AWS, “How S3 Versioning works.” https://docs.aws.amazon.com/AmazonS3/latest/userguide/versioning-workflows.html

[R13] AWS, “Locking objects with S3 Object Lock.” https://docs.aws.amazon.com/AmazonS3/latest/userguide/object-lock.html

[R14] OpenTelemetry, “Documentation.” https://opentelemetry.io/docs/

[R15] AWS, “Amazon RDS for PostgreSQL.” https://docs.aws.amazon.com/AmazonRDS/latest/UserGuide/CHAP_PostgreSQL.html

[R16] AWS, “Architect for AWS Fargate for Amazon ECS.” https://docs.aws.amazon.com/AmazonECS/latest/developerguide/AWS_Fargate.html

[R17] pdfplumber project documentation. https://github.com/jsvine/pdfplumber

[R18] PaddleOCR project documentation. https://github.com/PaddlePaddle/PaddleOCR

[R19] docTR project documentation. https://github.com/mindee/doctr